

National University of Computer and Emerging Sciences



Lab Manual 13 Artificial Intelligence

Course Instructor	Ms. Umm-e-Ammarah
Lab Instructor	Muhammad Hasan
Lab Instructor Email ID	m.hasan@lhr.nu.edu.pk
Semester	Spring 2026

FAST - School of Computing
FAST-NU, Lahore Campus

Contents

Lab Manual 13	3
Objectives:.....	3
Activities:.....	3
Activity 1 (Machine Learning Concepts):	3
Unsupervised Learning	3
• Algorithms	3
• Why Unsupervised Learning?	3
• Types of unsupervised learning.....	3
○ Clustering	3
▪ Types of Clustering.....	3
K-Means Clustering	3
• Using Built in Library	4
○ Import Libraries.....	4
○ Create sample data	4
○ Apply K-Means clustering	5
○ Visualize the clustered data	5
○ Visualize the Elbow Method	5
▪ Key points about inertia:	6
• From scratch	6
K-Medoids Clustering	7
• Using Built in Library	7
○ Import Libraries.....	7
○ Create sample data	7
○ Apply K-Means clustering	8
○ Visualize the clustered data	8
○ Visualize the Elbow Method	9
• From scratch	9
Activity 2 (Problem Statements / Tasks):.....	9
1. K-Means Clustering with Cost Analysis	9
2. Elbow Method for Optimal k.....	10
3. K-Means vs K-Medoids Comparison	10
4. Effect of Outliers on Clustering	11
5. Distance Metric Exploration	11

Lab Manual 13

Objectives:

- ✓ Understand unsupervised learning
- ✓ Perform k-means clustering
- ✓ Perform k-medoids clustering

Activities:

Activity 1 (Machine Learning Concepts):

Unsupervised Learning

Unsupervised Learning is a machine learning technique in which the users do not need to supervise the model. Instead, it allows the model to work on its own to discover patterns and information that was previously undetected. It mainly deals with the unlabelled data.

- **Algorithms**

Unsupervised Learning Algorithms allow users to perform more complex processing tasks compared to supervised learning. Although, unsupervised learning can be more unpredictable compared with other natural learning methods. Unsupervised learning algorithms include clustering, anomaly detection, neural networks, etc

- **Why Unsupervised Learning?**

Here, are prime reasons for using Unsupervised Learning:

- Unsupervised machine learning finds all kinds of unknown patterns in data.
- Unsupervised methods help you to find features which can be useful for categorization.
- It takes place in real time, so all the input data to be analyzed and labeled in the presence of learners.
- It is easier to get unlabeled data from a computer than labeled data, which needs manual intervention.

- **Types of unsupervised learning**

Here, are prime reasons for using Unsupervised Learning:

- *Clustering*

Clustering is an important concept when it comes to unsupervised learning. It mainly deals with finding a structure or pattern in a collection of uncategorized data. Clustering algorithms will process your data and find natural clusters(groups) if they exist in the data. You can also modify how many clusters your algorithms should identify. It allows you to adjust the granularity of these groups.

- **Types of Clustering**

- ✓ **Exclusive (partitioning):** In this clustering method, Data is grouped in such a way that one data can belong to one cluster only. Example: K-means
- ✓ **Agglomerative:** In this clustering technique, every data is a cluster. The iterative unions between the two nearest clusters reduce the number of clusters. Example: Hierarchical clustering

K-Means Clustering

K means it is an iterative clustering algorithm which helps you to find the highest value for every iteration. Initially, the desired number of clusters are selected. In this clustering method, you

need to cluster the data points into k groups. A larger k means smaller groups with more granularity in the same way. A lower k means larger groups with less granularity.

The output of the algorithm is a group of "labels." It assigns data points to one of the k groups. In **k-means clustering, each group is defined by creating a centroid for each group.** The centroids are like the heart of the cluster, which captures the points closest to them and adds them to the cluster.

The performance of K-Means is evaluated using a cost function, also known as the objective function or distortion function. It measures the total squared distance between each data point and the centroid of its assigned cluster. The algorithm aims to minimize this cost.

The K-Means cost function is defined as:

$$J = \sum_{n=1}^N \sum_{k=1}^K r_{nk} \|\mathbf{x}_n - \boldsymbol{\mu}_k\|^2$$

- Using Built in Library

Make sure Python and pip are already installed on your system.

- Import Libraries

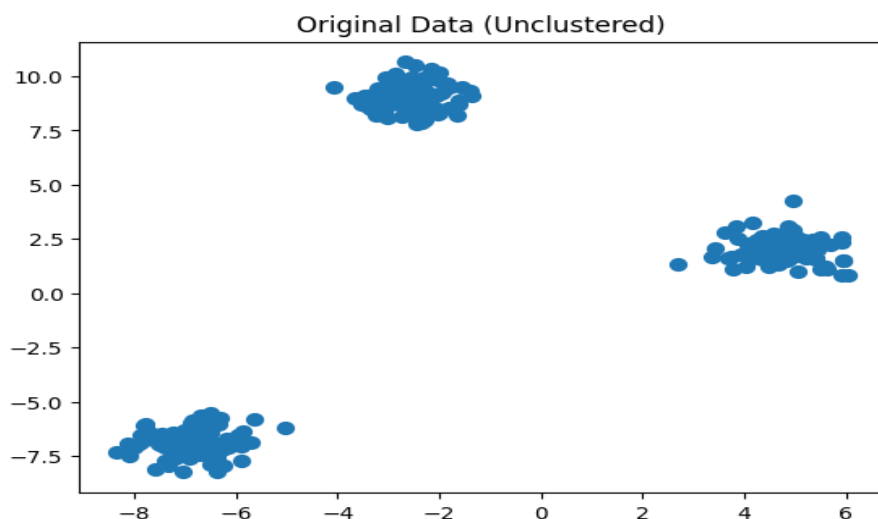
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs #to generate sample data
```

- Create sample data

We'll use make_blobs to create synthetic 2D data points.

```
# Create dataset with 3 centers (clusters)
X, y_true = make_blobs(n_samples=300, centers=3,
                        cluster_std=0.6, random_state=42)
```

```
# Visualize the raw data
plt.scatter(X[:, 0], X[:, 1], s=50)
plt.title("Original Data (Unclustered)")
plt.show()
```



- Apply K-Means clustering

```
# Create a KMeans instance with 3 clusters
kmeans = KMeans(n_clusters=3, random_state=42)
```

```
# Fit KMeans on the data
kmeans.fit(X)
```

```
# Get cluster centers and labels
centers = kmeans.cluster_centers_
labels = kmeans.labels_
```

- Visualize the clustered data

```
# Plot clusters
plt.scatter(X[:, 0], X[:, 1], c=labels, s=50, cmap='viridis')

# Plot cluster centers
plt.scatter(centers[:, 0], centers[:, 1], c='red', s=200,
alpha=0.7, marker='X')
plt.title("K-Means Clustering Results")
plt.show()
```



- Visualize the Elbow Method

Inertia measures how internally coherent the clusters are. It is essentially the same as the K-Means cost function, i.e., the sum of squared distances between each data point and the centroid of its assigned cluster.

Formally, inertia corresponds to:

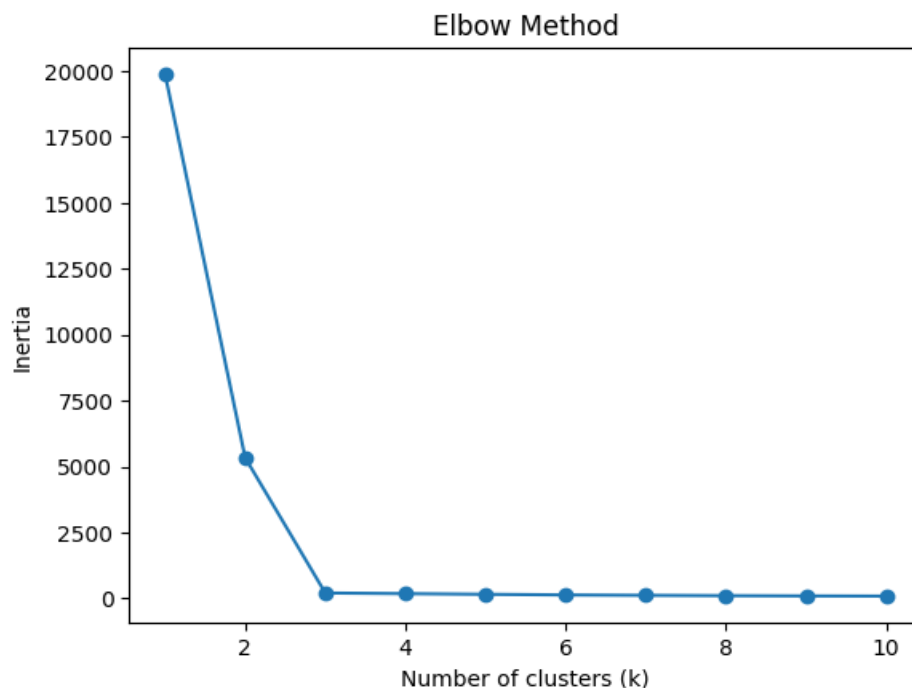
$$\text{Inertia} = \sum_{i=1}^k \sum_{x \in C_i} \|x - \mu_i\|^2$$

- Key points about inertia:
 - ✓ Inertia is not a normalized metric, so it depends on the scale of your data.
 - ✓ Lower inertia values indicate tighter clusters and better separation (but zero is only possible if every point is its own cluster).
 - ✓ In high-dimensional spaces, Euclidean distances can become inflated, this is part of the curse of dimensionality.
 - ✓ Running a dimensionality reduction technique such as PCA before applying K-Means can help mitigate this issue and speed up computation.

```
inertia = [] # Sum of squared distances to nearest cluster center

# Test k values from 1 to 10
for k in range(1, 11):
    km = KMeans(n_clusters=k, random_state=42)
    km.fit(X)
    inertia.append(km.inertia_)

# Plot the elbow curve
plt.plot(range(1, 11), inertia, marker='o')
plt.title("Elbow Method")
plt.xlabel("Number of clusters (k)")
plt.ylabel("Inertia")
plt.show()
```



- From scratch

Partial from scratch implementation of k-means is provided [here](#).

K-Medoids Clustering

K-Medoids is a clustering algorithm that groups data into k clusters, similar to K-Means. However, instead of using the mean (centroid) of points, it selects actual data points as cluster centers called medoids.

The most classical and widely used implementation of K-Medoids is the PAM (Partitioning Around Medoids) algorithm.

A medoid is the data point in a cluster whose total distance to all other points in the same cluster is minimum.

A larger k means smaller, more granular clusters and a smaller k mean larger clusters with less granularity.

PAM works in two main phases:

- BUILD Phase (Initialization)
 - ✓ Select the first medoid randomly.
 - ✓ Select the remaining medoids one by one.
 - ✓ At each step, choose the point that reduces the total clustering cost the most.

This creates a strong initial set of medoids (better than purely random selection).
- SWAP Phase (Optimization)
 - ✓ Assign each data point to the nearest medoid.
 - ✓ For each medoid m and non-medoid point o :
 - ✓ Swap m and o .
 - ✓ Compute new total cost.
 - ✓ If the swap reduces cost, keep it.
 - ✓ Repeat until no swap improves the solution.

This guarantees local optimal clustering.

PAM minimizes the total dissimilarity between points and their assigned medoid.

$$\sum_{j=1}^k \sum_{i \in C_j} d(x_i, z_j)$$

Unlike K-Means, this cost function requires no squared distances and works with any distance metric (Euclidean, Manhattan, cosine, etc.). Lower the cost, better will be clustering.

- Using Built in Library

You might need to `pip install scikit-learn-extra`

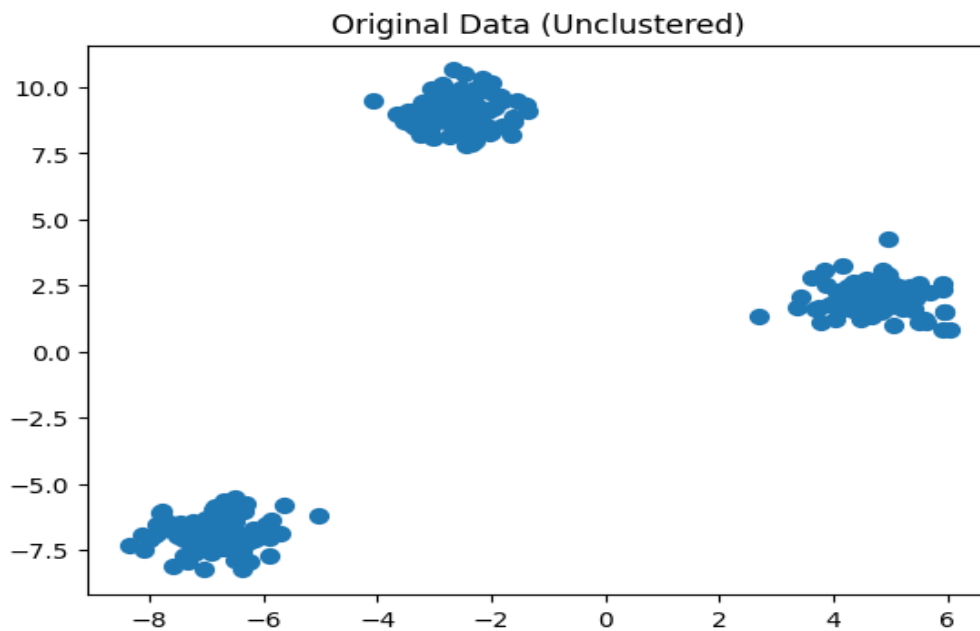
- Import Libraries

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn_extra.cluster import KMedoids
from sklearn.datasets import make_blobs
```

- Create sample data

We'll use `make_blobs` to create synthetic 2D data points.

```
X, y_true = make_blobs(n_samples=300, centers=3,
                        cluster_std=0.6, random_state=42)
plt.scatter(X[:, 0], X[:, 1], s=50)
plt.title("Original Data (Unclustered)")
plt.show()
```



- Apply K-Means clustering

```
kmedoids = KMedoids(n_clusters=3, method='pam', random_state=42)
```

```
kmedoids.fit(X)
```

```
medoids = kmedoids.cluster_centers_
```

```
labels = kmedoids.labels_
```

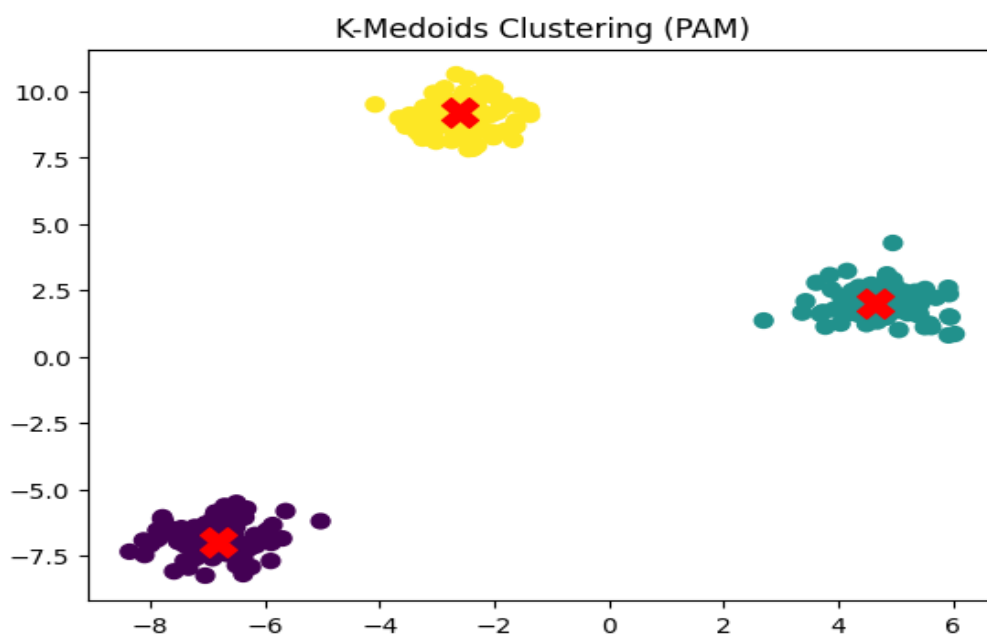
- Visualize the clustered data

```
plt.scatter(X[:, 0], X[:, 1], c=labels, s=50, cmap='viridis')
```

```
plt.scatter(medoids[:, 0], medoids[:, 1], c='red', s=200, marker='X')
```

```
plt.title("K-Medoids Clustering (PAM)")
```

```
plt.show()
```



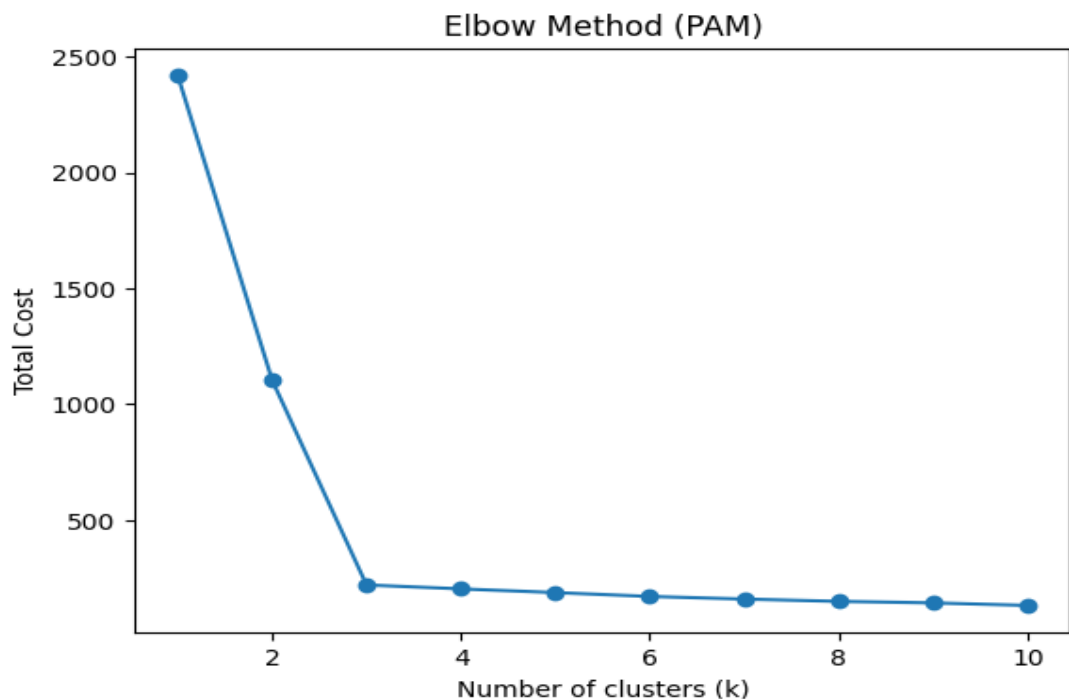
- Visualize the Elbow Method

Instead of inertia (squared distance), PAM uses total distance.

```
cost = []

for k in range(1, 11):
    model = KMedoids(n_clusters=k,
                    method='pam',
                    random_state=42)
    model.fit(X)
    cost.append(model.inertia_)

plt.plot(range(1, 11), cost, marker='o')
plt.title("Elbow Method (PAM)")
plt.xlabel("Number of clusters (k)")
plt.ylabel("Total Cost")
plt.show()
```



- From scratch

Explore the from scratch implementation of k-medoids.

Activity 2 (Problem Statements / Tasks):

1. K-Means Clustering with Cost Analysis

You are using a synthetic dataset generated using `make_blobs` with 300 samples and 3 centers.

Tasks:

- Apply K-Means clustering with:
 - $k = 3$

- Display:
 - Cluster labels
 - Cluster centroids
- Compute and display:
 - Inertia (cost function) of the model
- Visualize:
 - Clustered data with centroids

Extension:

- Change $k = 2$ and $k = 4$
- Compare inertia values

Analysis:

- Why does inertia decrease as k increases?
- Does lower inertia always mean better clustering?

2. *Elbow Method for Optimal k*

You are using a dataset generated using `make_blobs`.

Tasks:

- Run K-Means for:
 - $k = 1$ to 10
- Store inertia values
- Plot:
 - k vs inertia (Elbow graph)

Output:

- Identify:
 - Optimal number of clusters

Analysis:

- What is the “elbow point”?
- Why is it important?
- What happens if k is too large or too small?

3. *K-Means vs K-Medoids Comparison*

Using the same dataset.

Tasks:

- Apply:
 - K-Means clustering
 - K-Medoids clustering (PAM)
- Display:
 - Cluster centers (centroids vs medoids)
- Plot:
 - Both clustering results separately

Compare:

- Inertia (K-Means)
- Cost (K-Medoids)

Analysis:

- Why does K-Medoids use actual data points?
- Which algorithm is more robust to outliers and why?
- Which one is computationally expensive?

4. *Effect of Outliers on Clustering*

You are using a dataset using `make_blobs`.

Tasks:

- Add 5 extreme outliers manually
- Apply:
 - K-Means
 - K-Medoids

Display:

- Cluster assignments
- Centers/medoids
- Plots before and after adding outliers

Analysis:

- Which algorithm is more affected by outliers?
- Why does this happen mathematically?

5. *Distance Metric Exploration*

You are given a dataset.

Tasks:

- Apply K-Medoids using:
 - Euclidean distance
 - Manhattan distance

Display:

- Clustering results for both
- Total cost for each

Analysis:

- How does distance metric affect clustering?
- When should Manhattan distance be preferred over Euclidean?